



Intrusion Detection System

Snort IDS Lab.

By: Eng. Ahmed Fatouh
Abad Training Center.

Table of Contents

1. Introduction.....	4
2. Lab Setup and install snort	5
3. Incident Analysis using Snort.....	8
4. Live Analysis Using Snort.....	9
5. iptables and snort alerts.....	13
6. Outbound traffic using snort.....	15

Figure-list

<i>Figure 1 use the NAT Network Virtual Box</i>	<i>5</i>
<i>Figure 2 to make sure IP Address to VMs</i>	<i>6</i>
<i>Figure 3 simple ping test to make sure to VMs can Communicate with each other.</i>	<i>6</i>
<i>Figure 4 the port 22 , ssh listen</i>	<i>9</i>
<i>Figure 5 can connect to kali Linux using ssh from Ubuntu VM</i>	<i>9</i>
<i>Figure 6 snort live analysis</i>	<i>10</i>
<i>Figure 7 medusa options</i>	<i>11</i>
<i>Figure 8 snort configuration file</i>	<i>12</i>
<i>Figure 9 iptables result</i>	<i>13</i>
<i>Figure 10 iptables result after add the rule</i>	<i>14</i>

1. Introduction.

Intrusion Detection or Prevention Systems (IDS and IPS) are import and useful tools in any cyber security toolkit. Whether you work in offensive or defensive cyber security an understanding of what they are, and how they work is an import part of your capability as a cyber security specialist.

Snort is the foremost Open-Source Intrusion Detection System (IDS) in the world. Snort IPS uses a series of rules that help define malicious network activity and uses those rules to find packets that match against them and generates alerts for users.

Snort can be deployed inline to stop these packets, as well. Snort has three primary uses: As a packet sniffer like tcpdump, as a packet logger — which is useful for network traffic debugging, or it can be used as a full-blown network intrusion prevention system. Snort can be downloaded and configured for personal and business use alike.

2. Lab Setup and install snort

For this lab will use two VMs (Ubuntu 18.04.2 LTS , Kali Linux 2020)

Make Sure both VMs are up and running and can communicate with each other The easiest way to this is to ensure that both VMs are set to use the same network. As you will need internet access for some parts of this lab session I recommend having both VMs set to use the **NAT Network** This setting can be found in the VM settings under Network Adapter (see below for an example from Virtual Box).

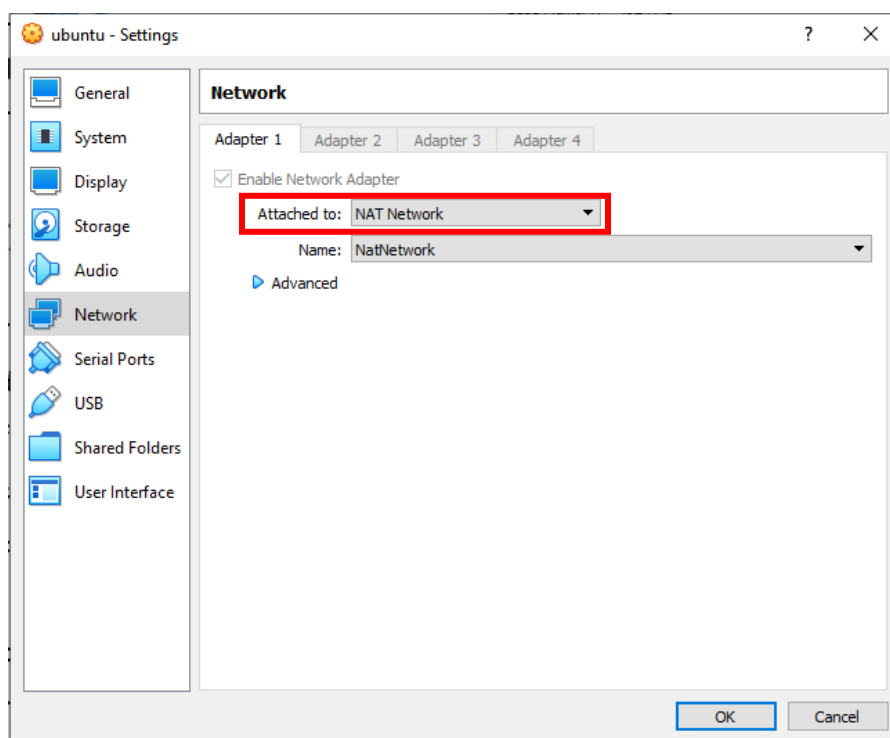
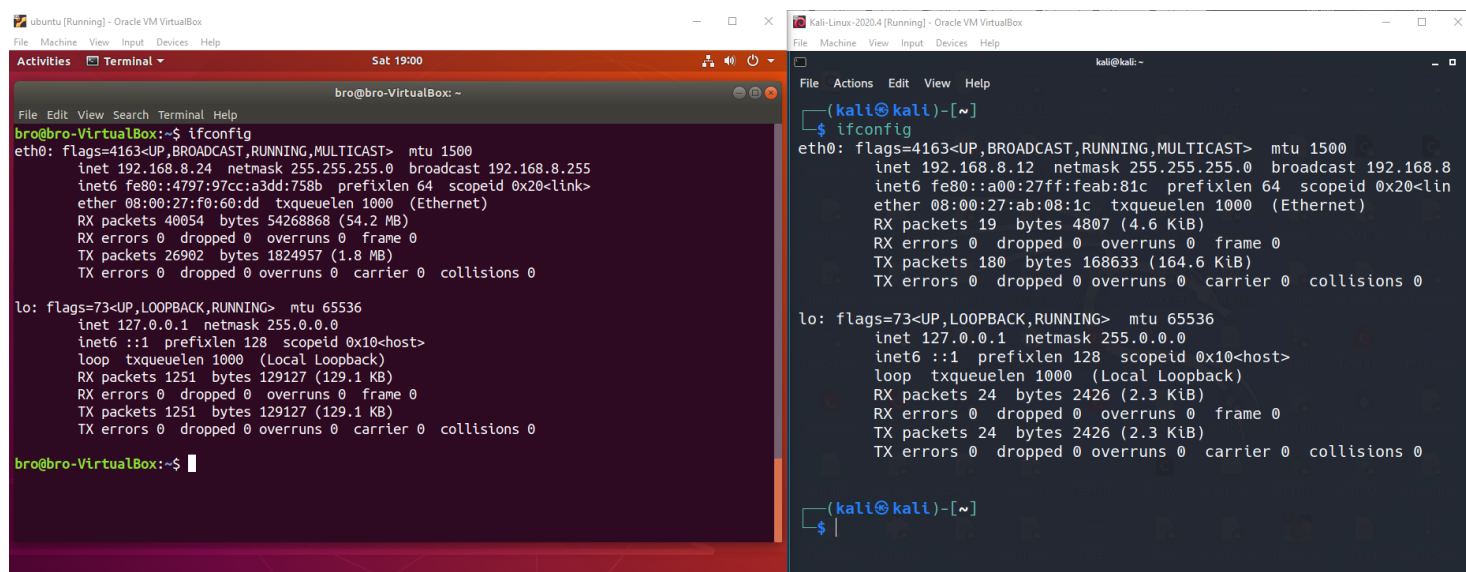


Figure 1 use the NAT Network Virtual Box

To make sure both VMs can communicate with each other we suggest a simple ping test (see below).

Ubuntu 18.04.2 LTS IP: 192.168.8.24/24

Kali Linux 2020 IP: 192.168.8.12/24



The image shows two side-by-side terminal windows. The left window is titled 'bro@bro-VirtualBox: ~' and shows the output of the 'ifconfig' command. The right window is titled 'kali@kali: ~' and also shows the output of the 'ifconfig' command.

```
bro@bro-VirtualBox:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.8.24 netmask 255.255.255.0 broadcast 192.168.8.255
    inet6 fe80::4797:97cc:a3dd:758b prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:f0:60:dd txqueuelen 1000 (Ethernet)
    RX packets 40054 bytes 54268868 (54.2 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26902 bytes 1824957 (1.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1251 bytes 129127 (129.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1251 bytes 129127 (129.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

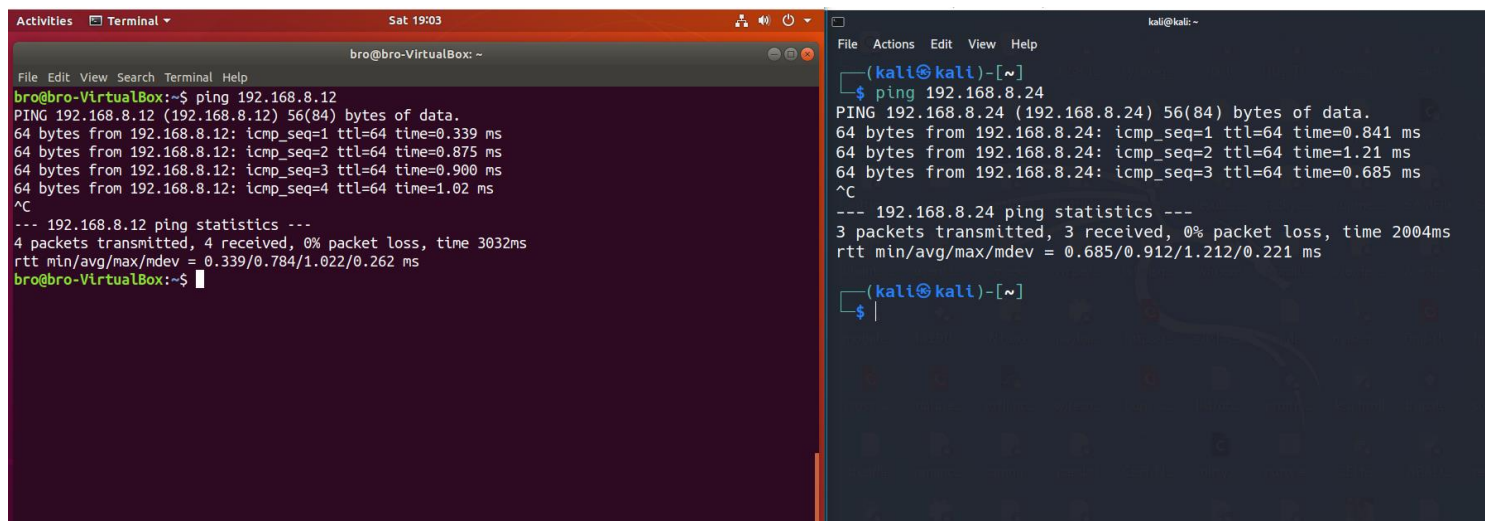
bro@bro-VirtualBox:~$
```

```
kali@kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.8.12 netmask 255.255.255.0 broadcast 192.168.8.255
    inet6 fe80::a00:27ff:feab:81c prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:ab:08:1c txqueuelen 1000 (Ethernet)
    RX packets 19 bytes 4807 (4.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 180 bytes 168633 (164.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 24 bytes 2426 (2.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 24 bytes 2426 (2.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

kali@kali:~$
```

Figure 2 to make sure IP Address to VMs



The image shows two side-by-side terminal windows. The left window is titled 'bro@bro-VirtualBox: ~' and shows the output of the 'ping 192.168.8.12' command. The right window is titled 'kali@kali: ~' and shows the output of the 'ping 192.168.8.24' command.

```
bro@bro-VirtualBox:~$ ping 192.168.8.12
PING 192.168.8.12 (192.168.8.12) 56(84) bytes of data:
64 bytes from 192.168.8.12: icmp_seq=1 ttl=64 time=0.339 ms
64 bytes from 192.168.8.12: icmp_seq=2 ttl=64 time=0.875 ms
64 bytes from 192.168.8.12: icmp_seq=3 ttl=64 time=0.900 ms
64 bytes from 192.168.8.12: icmp_seq=4 ttl=64 time=1.02 ms
^C
--- 192.168.8.12 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.339/0.784/1.022/0.262 ms
bro@bro-VirtualBox:~$
```

```
kali@kali:~$ ping 192.168.8.24
PING 192.168.8.24 (192.168.8.24) 56(84) bytes of data:
64 bytes from 192.168.8.24: icmp_seq=1 ttl=64 time=0.841 ms
64 bytes from 192.168.8.24: icmp_seq=2 ttl=64 time=1.21 ms
64 bytes from 192.168.8.24: icmp_seq=3 ttl=64 time=0.685 ms
^C
--- 192.168.8.24 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.685/0.912/1.212/0.221 ms
kali@kali:~$
```

Figure 3 simple ping test to make sure to VMs can Communicate with each other.

Once the VMs are setup and up and running, you have confirmed that they can communicate with each other NEXT Thing to do Install Snort In kali Linux VMs. To do that use the Command.

```
git clone https://github.com/plumpmonkey/SnortFastDeploymentScript.git
```

We recommend doing this in the Documents dir or somewhere else that is going to be easy to locate and access. Once the clone has completed cd into the newly created dir (SnortFastDeploymentScript) and run:

```
sudo ./SFDS
```

This will start the install process which will take around 10-15 minutes to complete.

3. Incident Analysis using Snort.

After install snort you should have PCAP file can download the pcap file onto kali Linux VM. (can get pcap file from this Link [PCAP File](#)).

Now open the DDOS_Attack.pcap using snort. To do this run the following command.

```
Snort -c /etc/snort/snort.conf -r DDOS_Attack.pcap -b -l .
```

This will load the pcap using Snort and generate a file called alert and a new pcap called snort.log (timestamp). Now open the alert file and the new pcap. Look at the same protocols that were identified as of interest during your manual examination.

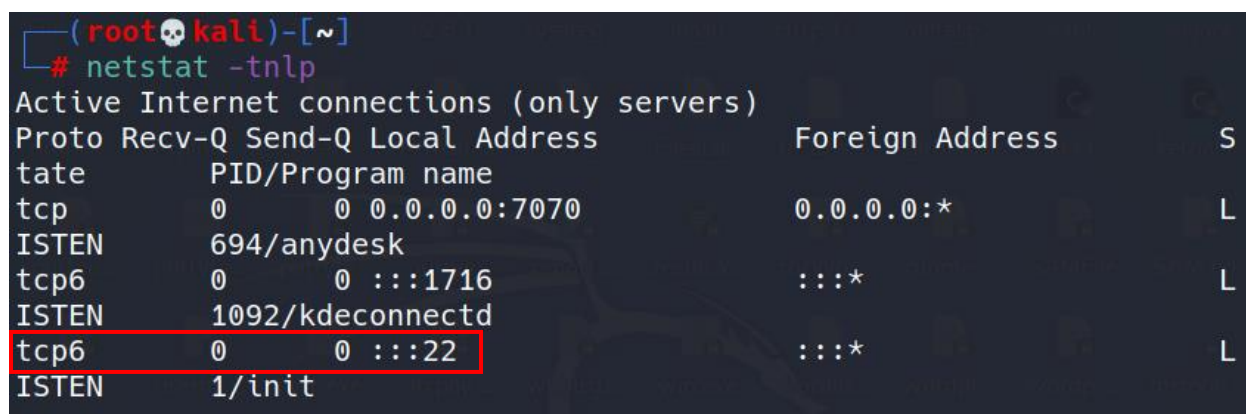
4. Live Analysis Using Snort.

The first thing to do will be to temporarily open a SSH socket on our Kali machine. To do this run the command:

```
sudo systemctl start ssh.socket
```

you confirm this has worked by running the command:

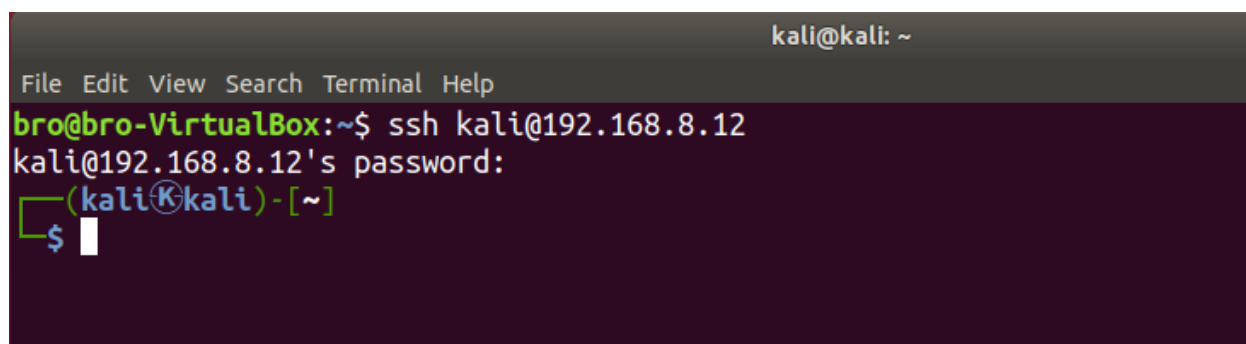
```
sudo netstat -tnlp | grep :22
```



```
(root@kali)-[~]
# netstat -tnlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         S
tate      PID/Program name
tcp        0      0 0.0.0.0:7070            0.0.0.0:*               L
ISTEN     694/anydesk
tcp6       0      0 :::1716                :::*                   L
ISTEN     1092/kdeconnectd
tcp6       0      0 :::22                  :::*                   L
ISTEN     1/init
```

Figure 4 the port 22 , ssh listen

Which should show that port 22 is set to LISTEN. You can then connect to your Kali VM from your Ubuntu VM using SSH.



```
kali@kali: ~
File Edit View Search Terminal Help
bro@bro-VirtualBox:~$ ssh kali@192.168.8.12
kali@192.168.8.12's password:
(kali@kali)-[~]
$
```

Figure 5 can connect to kali Linux using ssh from Ubuntu VM

Once you have confirmed that the SSH connection is working run the command:

```
sudo snort -A console -i eth0 -u snort -g snort -c /etc/snort/snort.conf
```

You should see Snort startup and start logging alerts and activity into the console

```

root@kali: ~
File Actions Edit View Help

Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Commencing packet processing (pid=1668)

03/20-13:41:57.672660  [**] [129:12:1] Consecutive TCP small segments
exceeding threshold [**] [Classification: Potentially Bad Traffic] [
Priority: 2] {TCP} 192.168.8.24:43512 -> 192.168.8.12:22
03/20-13:42:01.083304  [**] [1:1325:7] EXPLOIT ssh CRC32 overflow fil
ler [**] [Classification: Executable code was detected] [Priority: 1]
{TCP} 192.168.8.24:43514 -> 192.168.8.12:22
03/20-13:42:01.117116  [**] [129:12:1] Consecutive TCP small segments
exceeding threshold [**] [Classification: Potentially Bad Traffic] [
Priority: 2] {TCP} 192.168.8.24:43514 -> 192.168.8.12:22
|

```

Figure 6 snort live analysis

From the terminal of your Ubuntu VM

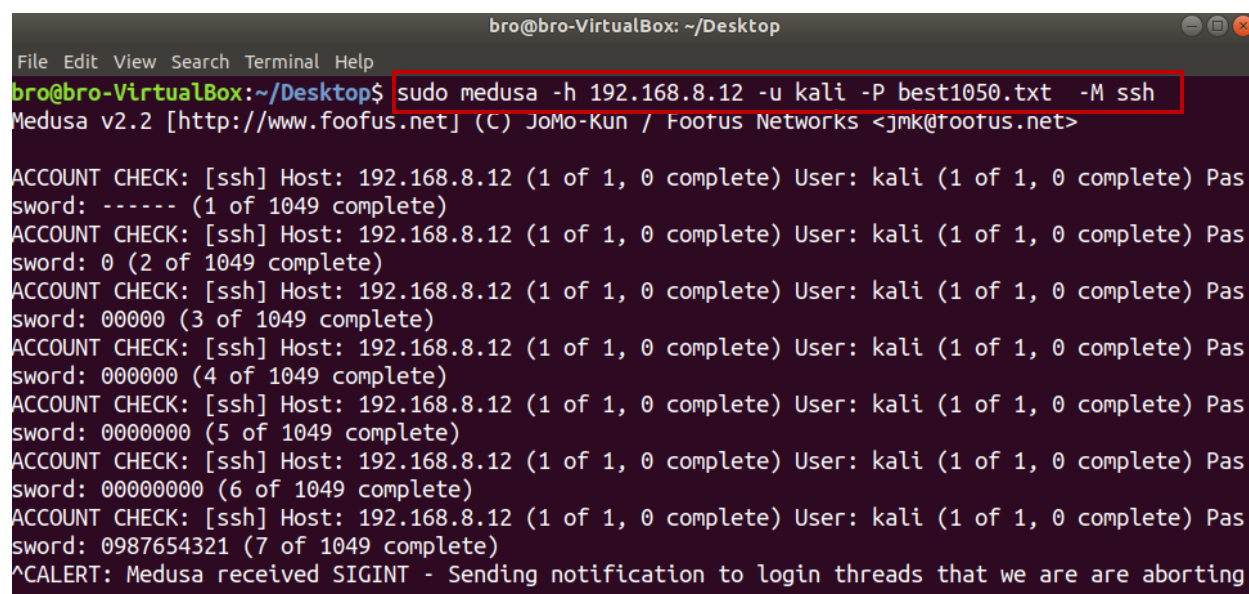
Can use Install Medusa , we will use medusa to brute force using wordlist (best1050.txt) can download the download from this Link [best1050.txt](#)

To install medusa in (Ubuntu VM) run the following command:

```
sudo apt-get install medusa
```

SSH brute force and a small wordlist. The object here is to demonstrate Snort in action. (to do SSH brute-force using medusa run the following command in Ubuntu VM).

```
sudo medusa -h 192.168.8.12 -u kali -P best1050.txt -M ssh
```

A screenshot of a terminal window titled 'bro@bro-VirtualBox: ~/Desktop'. The terminal shows the command 'sudo medusa -h 192.168.8.12 -u kali -P best1050.txt -M ssh' being executed. The output shows several 'ACCOUNT CHECK' messages for the user 'kali' on host '192.168.8.12', with passwords being tested one by one. The terminal also shows the Medusa version (v2.2) and the source (http://www.foofus.net). The command prompt is highlighted with a red box.

```
bro@bro-VirtualBox: ~/Desktop
File Edit View Search Terminal Help
bro@bro-VirtualBox:~/Desktop$ sudo medusa -h 192.168.8.12 -u kali -P best1050.txt -M ssh
Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Footus Networks <jmk@foofus.net>

ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: ----- (1 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 0 (2 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 000000 (3 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 000000 (4 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 00000000 (5 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 000000000 (6 of 1049 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.8.12 (1 of 1, 0 complete) User: kali (1 of 1, 0 complete) Pas
sword: 0987654321 (7 of 1049 complete)
^CALERT: Medusa received SIGINT - Sending notification to login threads that we are are aborting
```

Figure 7 medusa options

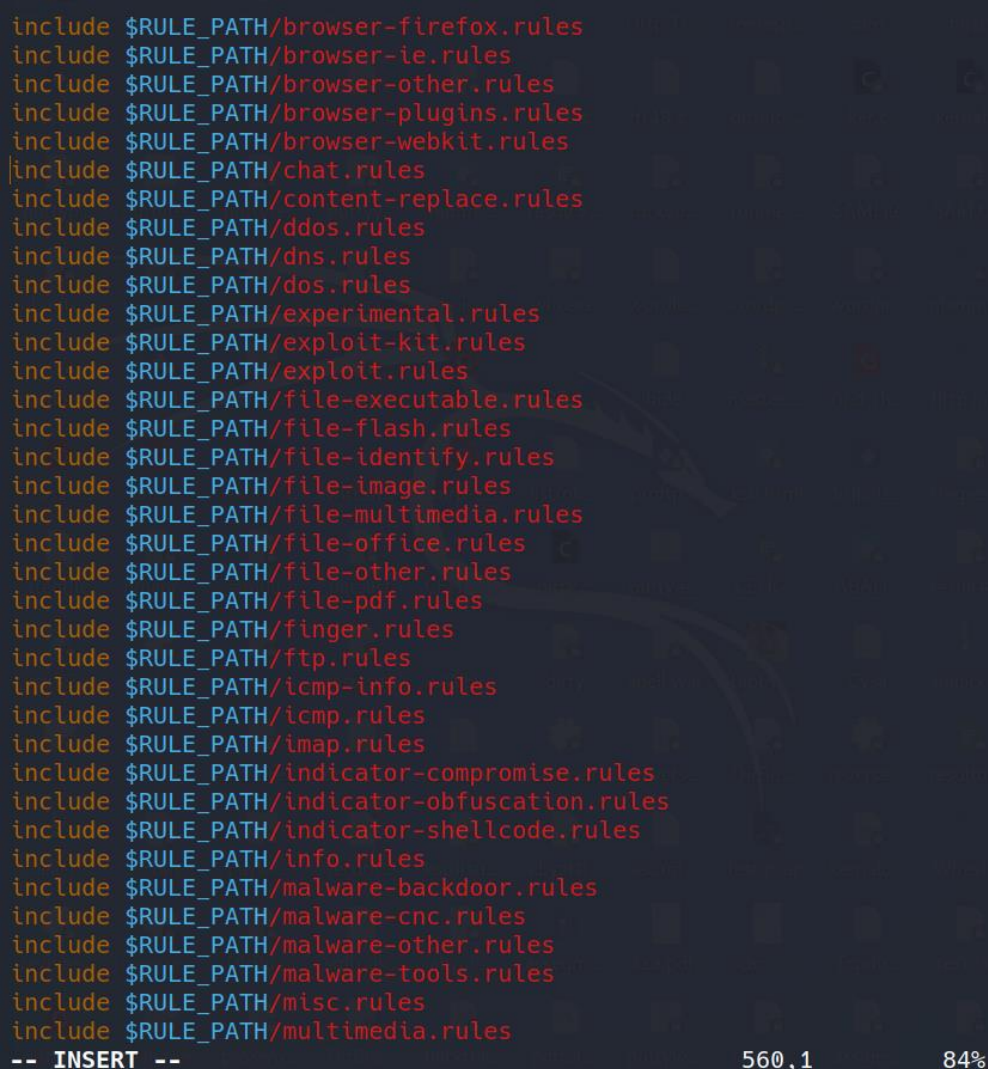
This will open the snort configuration file for editing - Because we used a tailored deployment for Snort here a lot of the rules are already setup for us...

In your kali Linux terminal run the following command:

```
sudo vim /etc/snort/snort.conf
```

find the section that has all the include \$RULE_PATH listings. There add in the line:

```
include $RULE_PATH/indicator-scan.rules
```



```
include $RULE_PATH/browser-firefox.rules
include $RULE_PATH/browser-ie.rules
include $RULE_PATH/browser-other.rules
include $RULE_PATH/browser-plugins.rules
include $RULE_PATH/browser-webkit.rules
include $RULE_PATH/chat.rules
include $RULE_PATH/content-replace.rules
include $RULE_PATH/ddos.rules
include $RULE_PATH/dns.rules
include $RULE_PATH/dos.rules
include $RULE_PATH/experimental.rules
include $RULE_PATH/exploit-kit.rules
include $RULE_PATH/exploit.rules
include $RULE_PATH/file-executable.rules
include $RULE_PATH/file-flash.rules
include $RULE_PATH/file-identify.rules
include $RULE_PATH/file-image.rules
include $RULE_PATH/file-multimedia.rules
include $RULE_PATH/file-office.rules
include $RULE_PATH/file-other.rules
include $RULE_PATH/file-pdf.rules
include $RULE_PATH/finger.rules
include $RULE_PATH/ftp.rules
include $RULE_PATH/icmp-info.rules
include $RULE_PATH/icmp.rules
include $RULE_PATH/imap.rules
include $RULE_PATH/indicator-compromise.rules
include $RULE_PATH/indicator-obfuscation.rules
include $RULE_PATH/indicator-shellcode.rules
include $RULE_PATH/info.rules
include $RULE_PATH/malware-backdoor.rules
include $RULE_PATH/malware-cnc.rules
include $RULE_PATH/malware-other.rules
include $RULE_PATH/malware-tools.rules
include $RULE_PATH/misc.rules
include $RULE_PATH/multimedia.rules
-- INSERT --
```

560,1 84%

Figure 8 snort configuration file

5. iptables and snort alerts.

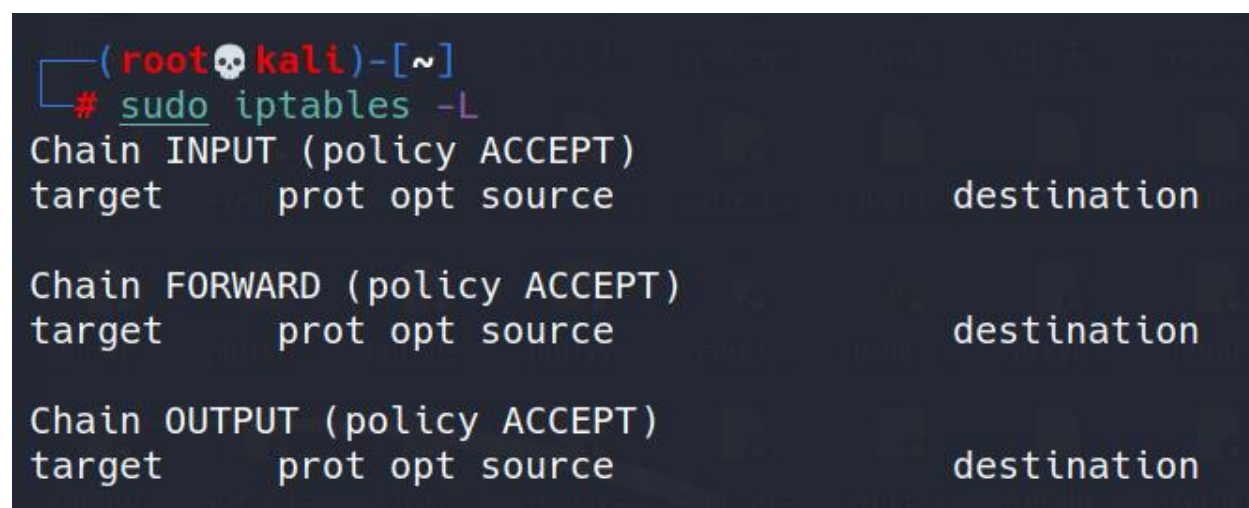
So we've got some useful information and we can quickly identify that something is happening - Now what?

Well one sensible approach might be to block the IP that is attacking us. Snort has made us aware of the problem, but we need to do something about it.

Enter **iptables**. We will only be using a couple of commands here.

In your kali Linux VM run the following commands:

```
sudo iptables -L
```



```
(root👤kali)-[~]  
# sudo iptables -L  
Chain INPUT (policy ACCEPT)  
target      prot opt source                destination  
  
Chain FORWARD (policy ACCEPT)  
target      prot opt source                destination  
  
Chain OUTPUT (policy ACCEPT)  
target      prot opt source                destination
```

Figure 9 iptables result

Next Run the following command:

```
sudo iptables -I Input -s 192.168.8.24 -J DROP
```

A terminal window with a dark background. The prompt is '(rootkali)-[~]'. The command '# sudo iptables -L' has been executed. The output shows the 'Chain INPUT (policy ACCEPT)' section. Below this, there is a table with headers: 'target', 'prot', 'opt', 'source', and 'destination'. The first row of data is 'DROP', 'all', '--', '192.168.8.24', and 'anywhere'. The 'DROP', 'all', '--', and '192.168.8.24' columns are enclosed in a red rectangular box.

```
(rootkali)-[~]  
# sudo iptables -L  
Chain INPUT (policy ACCEPT)  
target      prot opt source      destination  
DROP        all  --  192.168.8.24  anywhere
```

Figure 10 iptables result after add the rule

To remove the rule we put in place run the following:

```
sudo iptables -F
```


6. Outbound traffic using snort.

So that is inbound traffic covered - What about outbound traffic? Are there situations where we might want to keep any eye on outbound traffic? Of course. Users and / or employees visiting dangerous sites, insider threats and naturally malware.

It has things like white and black list rules. Let's look at the blacklist rules. Open up the blacklist rules using vim with:

```
sudo vim /etc/snort/rules/blacklist.rules
```

```
#-----
# BLACKLIST RULES
#-----
#
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain hesneclimi.ru - Packed.Win32.Krap.ae"; flow:to server; byte test:1,!&64,2;
byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[0A]hesneclimi[02]ru"; metadata:impact_flag red, service dns; reference:url,labs.snort.org/
docs/16887.html; classtype:trojan-activity; sid:16887; rev:3;)
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"BLACKLIST URI request for known malicious URI - /message.php?subid="; flow:to server,established;
content:"/message.php?subid="; nocase; http_uri; content:"version=nn2"; nocase; http_uri; metadata:impact_flag red, service http; reference:url,labs.snort.org/
docs/16925.html; classtype:trojan-activity; sid:16925; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain qd.netkill.com.cn - Trojan-Downloader.Win32.Adload.rzx"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[02]qd[07]netkill[03]com[02]cn"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16834.html; classtype:trojan-activity; sid:16834; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain sx21.e4578.com - Trojan.Win32.Scar.ccqb"; flow:to server; byte test:1,!&64,2;
byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[04]sx21[05]e4578[03]com"; metadata:impact_flag red, service dns; reference:url,labs.snort.org/
docs/16898.html; classtype:trojan-activity; sid:16898; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain members.multimania.co.uk - Trojan.Win32.Inject.ahqv"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[07]members[0A]multimania[02]co[02]uk"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16866.html; classtype:trojan-activity; sid:16866; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain bedayton.com - Trojan-Downloader.Win32.Agent.dlhe"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[08]bedayton[03]com"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16846.html; classtype:trojan-activity; sid:16846; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain postmetoday.ru - Packed.Win32.Katusha.j"; flow:to server; byte test:1,!&64,2;
byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[0B]postmetoday[02]ru"; metadata:impact_flag red, service dns; reference:url,labs.snort.org/
docs/16872.html; classtype:trojan-activity; sid:16872; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain hl.ripway.com - Trojan.Win32.Refroso.bcdq"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[02]hl[06]ripway[03]com"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16889.html; classtype:trojan-activity; sid:16889; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain ra03.e5732.com - Trojan-Clicker.Win32.Small.afg"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[04]ra03[05]e5732[03]com"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16836.html; classtype:trojan-activity; sid:16836; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain andy.cd - Backdoor.Win32.Agent.auto"; flow:to server; byte test:1,!&64,2;
byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[04]andy[02]cd"; metadata:impact_flag red, service dns; reference:url,labs.snort.org/
docs/16856.html; classtype:trojan-activity; sid:16856; rev:3;)
alert udp any 53 -> $HOME_NET any (msg:"BLACKLIST DNS request for known malware domain phaizeipeu.ru - Packed.Win32.Krap.gx"; flow:to server; byte test:1,!&64,2;
byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[0A]phaizeipeu[02]ru"; metadata:impact_flag red, service dns; reference:url,labs.snort.org/
docs/16862.html; classtype:trojan-activity; sid:16862; rev:3;)
alert udp $HOME_NET any -> any 53 (msg:"BLACKLIST DNS request for known malware domain livetrust.info - Trojan-Spy.Win32.Zbot.akkku"; flow:to server;
byte test:1,!&64,2; byte test:1,!&32,2; byte test:1,!&16,2; byte test:1,!&8,2; content:"[09]livetrust[04]info"; metadata:impact_flag red, service dns;
reference:url,labs.snort.org/docs/16907.html; classtype:trojan-activity; sid:16907; rev:3;)
```

Figure 11 blacklist in snort

We can use that behavior to get Snort to alert us to any malicious activity.

```
nslookup phaizeipeu.ru
```